Back

Practice Quiz: Behaviors
Practice Assignment • 30 min

English

Your grade: 100%

Your latest: **100%** • Your highest: **100%** • To pass you need at least 80%. We keep your highest score.

Next item

1. In an ecosystem simulation developed using Processing with Python mode, agents are programmed to exhibit various behaviors, one of which includes moving towards a target. Each agent and target in the simulation is represented by a position vector, indicating their location within the environment. Considering this setup, how can an agent be programmed to move towards a target effectively? 1 / 1 point

- ○
```
agent.position = target.position
```
- ◉
```
direction = PVector.sub(target.position, agent.position)
direction.normalize()
direction.mult(movementSpeed)
agent.position.add(direction)
```
- ○
```
agent.position.x += random(-1, 1)
agent.position.y += random(-1, 1)
```
- ○
```
if agent.position.x < target.position.x:
    agent.position.x += 1
else:
    agent.position.x -= 1

if agent.position.y < target.position.y:
    agent.position.y += 1
else:
    agent.position.y -= 1

```
- ○
```
if PVector.dist(agent.position, target.position) > minDistance:
    # Wait for the target to come closer
    pass

```

Correct
Correct. This approach effectively calculates a direction vector pointing from the agent to the target by subtracting the agent's position from the target's position. Normalizing this direction vector and then scaling it by the agent's movement speed ensures that the agent moves towards the target at a consistent speed. Finally, adding this scaled direction vector to the agent's current position simulates realistic movement towards the target.

2. In an ecosystem simulation using Processing with Python mode, each agent is designed to have a finite lifespan to simulate natural life cycles. The lifespan of an agent decreases over time until it reaches zero, at which point the agent is considered to have reached the end of its life. Given this setup, how can you implement a mechanism to simulate the decreasing lifespan of an agent effectively? 1 / 1 point

- ◉
```
agent.lifespan -= 1
if agent.lifespan <= 0:
    # Handle the end of the agent's life

```
- ○
```
agent.lifespan = 0
```
- ○
```
if random(0, 100) < deathChance:
    # Handle the end of the agent's life
```
- ○
```
agent.lifespan += agent.performAction()
```
- ○
```
if not agent.isSleeping:
    agent.lifespan -= 1
```

Correct
Correct. This approach effectively simulates the agent's lifespan by decrementing a lifespan counter each frame (or simulation step), which represents the passage of time. When the lifespan counter reaches or falls below zero, the agent has reached the end of its life, triggering any necessary actions to remove the agent from the simulation or handle its death. This method provides a simple yet realistic way to simulate aging and the lifecycle of agents in the simulation.

3. In a simulation environment using Processing with Python mode, agents are designed to interact with their surroundings, which include predators. To ensure survival, an agent must incorporate a behavior that allows it to stay away from a predator. Considering each agent and predator is represented by a position vector indicating their location in the environment, what strategy can an agent employ to effectively maintain a safe distance from a predator? 1 / 1 point

- ○
```
direction = PVector.sub(predator.position, agent.position)
agent.position.add(direction)
```
- ◉
```
direction = PVector.sub(agent.position, predator.position)
direction.normalize()
direction.mult(escapeSpeed)
agent.position.add(direction)
```
- ○
```
# Agent attempts to hide by remaining stationary
agent.velocity = PVector(0, 0)
```
- ○
```
agent.position.add(PVector(random(-1, 1), random(-1, 1)))
```
- ○
```
if PVector.dist(agent.position, predator.position) > safeDistance:
    agent.position.add(PVector.sub(predator.position, agent.position).normalize())
```

Correct
Correct. This solution effectively calculates the direction to move away from the predator by subtracting the predator's position from the agent's position, normalizing this direction vector, and then moving in that direction at a specified speed (escapeSpeed). This behavior allows the agent to increase the distance from the predator, effectively implementing an avoidance strategy.